

Interview Prep — Round 2

Delivery Manager – Engineering | Principal Global Services | Portfolio Manager Round

REVISED — People-leadership & conflict/scenario focus

THE ONE IDEA TO CARRY INTO THE ROOM

“I’ve been leading this team for two years, and I grew most of them myself. The job now is to make them excellent without me in the code.”

Every answer should ladder back to some version of that line. It is the antidote to the single biggest doubt in this round: *is this a manager, or an IC who wants the title?*

Read this first — what changed and why

Round 1 cleared you on track record. This round decides one thing: **can a senior leader picture you running this team without being in the weeds**. Your stack and your job are literally the job description — so the conversation will drift toward technical territory where you’re strong. **That drift is the trap**. Every minute on architecture is a minute not spent proving you can lead people. This doc is weighted to pull you back to people and conflict — that’s where the round is won or lost.

YOUR THROUGH-LINE STRENGTHS (LEAD WITH THESE)

You grew the team from 1 to 11 — mostly trainees, only 2–3 lateral hires. Most managers inherit a team. You built one. This is your single most powerful asset in this round — use it early and often.

You carried that team through repeated technology reinvention (Scala Spark → PySpark, on-prem → AWS, Snowflake adoption) without losing people or delivery. That’s change leadership, not just migration.

Anti-rambling framework (unchanged — it works)

The 3-beat story — 60–90 seconds, never past 2 minutes

- **Beat 1 — Setup (15s).** One sentence on the situation, one on what made it hard. Resist adding context.
- **Beat 2 — What you did (30–45s).** Two or three decisive moves *you* personally made. Not a chronology. Pick the moves a manager would care about — usually the people moves.
- **Beat 3 — Outcome + insight (15–30s).** Measurable result + one sentence on what you learned or now do differently. Then stop.

Headline first, always. Open with the punchline, then unpack. Never build up to it.

THREE PHRASES THAT BUY YOU TIME

“Let me think about that for a second.” — makes you sound considered, not stuck.

“The short answer is X. The longer answer...” — headline up front, then elaborate without rambling.

“Are you more interested in the technical side of that, or the people side?” — narrows the question

and steers you to the right story. Use this a lot in this round.

Part A — Your people-leadership story arsenal

Force the ratio: walk in with **three people stories and one technical story**, not four technical and two blanks. Demote Snowflake-vs-SageMaker and the migration to “only if directly asked on the tech.” Lead with the three below.

Story 1 (flagship) — You built the team, 1 → 11

Headline: “I grew this team from one person to eleven — and most of them I brought in as trainees and developed into independent engineers. Only two or three were lateral hires.”

Say it like this

“When I started, it was me. Today it’s a team of eleven, and the thing I’m proudest of is that most of them joined as trainees — I hired for raw ability and grew them into engineers who now own production systems independently.

Three things made that work. First, I gave people real ownership early — not toy tasks, but a feature pipeline they were accountable for, with me as a safety net rather than a co-pilot. Second, I paired deliberately: I’d sit with someone on the hard parts, then deliberately step back on the next one so they’d stretch. Third, I protected their learning curve from delivery pressure — I absorbed the ‘why isn’t this done yet’ so they had room to actually learn.

The outcome is a team that now runs feature delivery without me in the critical path — which is exactly the test of whether I built people or just built software. What I learned: the multiplier isn’t my output, it’s how fast I can make someone else independent.”

WHY THIS IS YOUR BEST STORY

It proves the exact thing the round is testing — that you make *other* people deliver — and it’s unarguably true and specific. It also quietly answers “are you really a manager?” before they ask. If you only nail one story tomorrow, nail this one.

SHARPEN IT TONIGHT (OPTIONAL, 5 MIN)

Pick ONE trainee as a mini-example inside this story: who they were when they joined, the one thing you did, where they are now. One named arc makes the whole story land harder than the aggregate.

Story 2 — Leading a team through repeated technology shifts

Headline: “I’ve taken this team through three major technology shifts — Scala Spark to PySpark, on-prem to AWS, and adopting Snowflake — without losing momentum or people. The hard part was never the tech; it was the team’s confidence.”

Say it like this

“We’ve reinvented our stack more than once — Scala Spark to PySpark on AWS, and standing up Snowflake. With a team I’d grown in-house, the real risk wasn’t the migration mechanics, it was people feeling their hard-won expertise had just been reset to zero.

So I led it as a people problem first. I sequenced the shift so no one was thrown in cold — I went first on the riskiest piece, turned it into a worked example, then had the team learn on lower-stakes workloads before the critical ones. I framed every shift as ‘you’re becoming more valuable,’ not ‘relearn everything.’ And I set up benchmarks so we could prove to ourselves — and to stakeholders — that the new stack was actually better, not just newer.

We came through each shift with the team intact and delivery uninterrupted. What I took from it: when you're changing the ground under people, your job is to protect their confidence as carefully as you protect the timeline."

FRAMING NOTE

This is your migration story, but told as **change leadership**, not architecture. If they push on the technical detail, give it briefly — then steer back: "...but the part I'm proudest of is that the team came through it stronger."

Story 3 — A real disagreement: Snowflake vs. SageMaker Feature Store

Headline: "One of our architects made a genuinely strong case for SageMaker Feature Store. He wasn't wrong. We went with Snowflake anyway — and I made sure he understood why, so the decision had his buy-in, not just my sign-off."

Say it like this

"During the feature-store evaluation, one of our architects argued for SageMaker Feature Store. And he had a real case — on pure ML-native capability, it was a defensible choice. So this wasn't a weak argument I could wave away; reasonable engineers disagreed.

I handled it by separating the technical merit from the business decision. I acknowledged openly that on the narrow ML criteria he was right. But we already had a significant footprint of data products in Snowflake, and choosing it gave us far stronger governance and a single place to manage and secure features. So I laid the trade-off out explicitly — ML-native depth versus governance and consolidation — and made the call on the business axis, not the technical one.

The outcome that matters isn't just that we shipped on Snowflake — it's that the architect was on board, because I engaged with his argument instead of overriding it. What I learned: when someone smart disagrees, the win isn't proving them wrong, it's making sure the final call is one they can stand behind."

WHY THIS LANDS WITH A PORTFOLIO MANAGER

It shows three senior traits at once: you **concede the other side had merit** (secure, not defensive), you **decide on business value not technical preference** (manager thinking, not IC thinking), and you **preserve the relationship**. That last part is the whole game in a portfolio role.

Story 4 (hold in reserve) — The Feature Store, told as a team/process win

Use only if asked for a delivery example. Headline: "I led the design and delivery of our centralized Feature Store on AWS Glue and Snowflake — it now powers 10+ ML models. But the lasting win is the operating rhythm I built around it."

- **Pivot to people fast:** "The architecture mattered, but what I'm proudest of is the sprint cadence and quality gates the team still runs on today — that's the part that outlives any one platform."

DISCIPLINE

Do not lead with this. Every extra minute on the architecture reinforces the IC image. One crisp telling, then back to people.

Part B — Scenario & conflict playbook (the questions you had no answer for)

These are the highest-risk questions for a first-time manager, and your original doc had nothing for them. Don't script these word-for-word — internalize the **structure**, so you can apply it to whatever they actually ask.

B1 — “Tell me about a time you managed an underperformer.”

THIS IS THE QUESTION FOR A FIRST-TIME MANAGER

They ask it because conflict-averse new managers let weak performers coast and the team rots. “I’ve never had to” — said flatly — is a **yellow flag**. Below is how to answer truthfully without exposing yourself.

Your honest, strong version (say it like this)

“I’ll be straight with you — I haven’t had to formally manage someone out, and there’s a reason: most of my team I grew from trainees, so I was shaping performance early and continuously rather than letting gaps harden into a crisis. Catching it early is the management.

Where I have acted is on early underperformance signals — someone whose delivery was slipping or whose quality wasn’t at the bar. My approach is the same every time: I make the expectation explicit and specific, I get curious before I get critical — is this a skill gap, a clarity gap, or something going on outside work — and then I give them a concrete, time-boxed path to close it, with support attached.

And I’m clear-eyed about the harder version: if someone genuinely can’t meet the bar after real support, protecting the rest of the team means acting, not avoiding. I haven’t had to go there, but I won’t flinch from it if the situation calls for it.”

The structure underneath (memorize this, not the words)

1. **Clarify** — make the expectation explicit and specific; vague standards cause half of all “underperformance.”
2. **Diagnose** — skill gap vs. clarity gap vs. personal/external. The fix differs for each.
3. **Support + time-box** — concrete improvement plan, real help, a clear window.
4. **Decide** — if it doesn’t move, act, because the cost of a tolerated underperformer is paid by the whole team.

B2 — “Tell me about a time you gave difficult feedback.”

SCAFFOLD — FILL ONE REAL MOMENT IN TONIGHT

You told me you rarely needed *hard* corrective feedback — fine. But you gave **developmental** feedback constantly while growing trainees. That counts, and it’s the honest version. Drop one real instance into the brackets below.

Frame + fill-ins

“Most of my feedback has been developmental rather than disciplinary, because I was growing people — but some of it was hard to give. For example, *[name the person/role — e.g., a strong*

coder who kept over-engineering, or someone whose work was good but who went quiet in stakeholder forums].”

“The gap was [what specifically wasn’t working — e.g., ‘their solutions were technically elegant but consistently missed the deadline because they gold-plated’]. I raised it directly but privately, and I framed it around their growth, not a complaint: [roughly what you said — e.g., ‘your engineering is strong enough that the thing holding you back now is judgment on when good-enough is the right call’].”

“What changed: [the result — e.g., ‘they started time-boxing and shipping, and within a couple of months they were the one mentoring others on scoping’]. What I learned about my own style: [one line — e.g., ‘feedback lands when it’s about unlocking the person, not correcting them’].”

IF YOU TRULY CAN’T FIND ONE

Use this honest pivot: “The hardest feedback I give is telling a strong engineer that the skill holding them back now isn’t technical — it’s *[scoping / communication / delegation]*. That’s hard because they’re used to being praised for the technical side.” Then describe how you handle that conversation in general. It’s real, and it shows you give feedback thoughtfully.

B3 — “How do you lead people who were your peers yesterday?”

This is your specific situation — they will likely ask it. Don’t pretend the dynamic doesn’t change.

“I’d name it rather than pretend it isn’t happening. I’d be transparent with the team that the relationship is shifting, and that my job now is to clear the path for them, not to suddenly start bossing people I respect.

Practically: I earn the authority instead of asserting it. I keep doing the things that built their trust — being in the detail enough to be useful, being honest, taking heat for them — and I add the new things the role needs: owning their growth, making the priority calls, having the harder conversations when they’re needed. Most of my team I grew anyway, so the trust is already there; I just have to be careful not to spend it carelessly in the first few weeks.”

B4 — Competing priorities & organizational ambiguity

“How do you handle competing priorities from multiple stakeholders?”

“My default is to make the trade-off visible rather than silently absorb it. When I have three asks and capacity for two, I lay out what each delivers, what it costs, and what gets deferred — and I take that to whoever owns prioritization, with my recommendation attached. I don’t try to quietly do all three, because silent over-commitment destroys team trust faster than any single missed date.”

“What do you do when priorities are unclear and nobody’s deciding?”

“I create clarity instead of waiting for it. I’ll write down the two or three plausible interpretations of what we should be doing, attach my recommended one with the reasoning, and take it to the people who can confirm or correct. Forcing a concrete proposal usually unsticks a stalled decision — people react to a draft far faster than to an open question. A manager who freezes without direction is a risk; I’d rather be the one who proposes the direction.”

B5 — Exec-level communication (a slipping delivery)

“How would you tell my boss that a delivery is slipping?” They’re testing whether you can compress and lead with the ask — the same muscle as your anti-rambling work.

THE FOUR-LINE EXEC UPDATE — USE THIS SHAPE

Headline: “The X delivery is at risk of slipping two weeks.” (Lead with it. Never bury it.)

Impact: “Here’s what that affects downstream.”

Action: “Here’s what I’m already doing about it.”

Ask: “Here’s the one decision or unblock I need from you.”

Say: “I brief up the way I’d want to be briefed — headline first, then impact, then what I’m doing, then the one thing I need. I never make a senior leader dig for the point, and I never bring a problem without a proposed path. Escalating early and cleanly is a strength, not an admission of failure.”

B6 — Headcount & budget thinking

“One hire: a senior engineer or two juniors — how do you decide?” You’re moving into a role that owns a cost center. Show you treat the team as an investment.

“It depends on what the team needs structurally, not just who’s available. I’d ask: is the bottleneck a lack of senior judgment — architecture, mentoring, unblocking — or a lack of hands to execute well-defined work? If it’s judgment, one senior is worth more than two juniors. If it’s throughput on clear work, and the team has enough senior coverage to mentor, two juniors give me more capacity *and* a pipeline I can grow — which, given my track record, is something I’m good at.

Honestly, my bias is toward growing talent — I’ve built most of this team that way and it compounds. But I wouldn’t add juniors if there’s no senior bandwidth to develop them, because then I’m just adding load. The decision is: where’s the real constraint, and which option returns the most value over the next year.”

THE MINDSET TO SIGNAL

A team is an investment that has to return value. You can reason about composition (senior vs. junior, build vs. buy), and you tie headcount to the team’s actual constraint — not to empire-building.

B7 — Strategic thinking across portfolios

“How does your team’s work create leverage beyond your team?” You are sitting on the perfect answer and your old doc never framed it strategically.

“The feature store isn’t really a project for my team — it’s shared infrastructure for the org. The whole point of a centralized, governed feature store is that features built once get reused across many models and, potentially, many teams. So the way I think about it is: every feature we standardize is leverage someone else doesn’t have to rebuild.

That changes how I’d prioritize as a manager — I’d weight work partly by how reusable it is, not just by my own team’s immediate backlog. And I’d actively look for the teams beyond mine that could consume it, because that’s where a feature store goes from a team asset to a portfolio asset.”

THIS SINGLE REFRAME DOES A LOT

It proves you think past your own box — the exact “do you see beyond your team” test — and it recasts your most technical achievement as org-level strategy. Work a version of this into the conversation even if they don’t tee it up.

Part B+ — More manager-judgment questions

Three more a Portfolio Manager is likely to use to see whether you reason like a manager. Hold the **structure**, not a script.

N1 — “How would you compare two teams and tell which one is high-performing?”

WHAT THEY'RE REALLY TESTING

Your **bar for excellence**. The trap is answering with velocity alone — “the one that ships more.” A senior leader wants to hear that you judge a team on more than output, and that you’d distrust a team that looks fast but is fragile.

Say it like this

“I wouldn’t judge it on raw output — the team that ships the most isn’t automatically the better team. I’d compare on four things.

First, predictability — does the team do what it said it would, sprint after sprint? Consistency matters more to me than occasional bursts. Second, quality — what does it cost them after they ship: rework, incidents, escaped defects? A team that ships fast and firefights constantly is borrowing against the future. Third, growth — are the people getting better, are juniors becoming independent, is the team’s ceiling rising? Fourth, resilience — can it deliver when the manager or a key person is out? A team that only works because one hero holds it together isn’t high-performing, it’s exposed.

So the high-performing team is the one that’s predictable, ships clean, grows its people, and doesn’t depend on any single person — including me. A team that looks fast on a burndown but fails three of those four isn’t the one I’d bet on.”

IF THEY PUSH: “GIVE ME ONE METRIC.”

Don’t take the bait for a single vanity metric. Say: “If I had to pick one signal, it’s **predictability — committed vs. delivered over time** — because a team that consistently hits its word usually has the quality and the planning discipline underneath it. But I’d never look at it without the rework and attrition numbers next to it.”

N2 — “Two people on your team expect promotion at the same time. How do you handle it?”

WHAT THEY'RE REALLY TESTING

Whether you can be **fair, transparent, and decisive** under emotional pressure — and crucially, how you handle the person who *doesn’t* get it. Dodging (“I’d promote both”) or vagueness is the fail. Retaining the disappointed one is the real test.

Say it like this

“First, I separate expectation from readiness. Both expecting it doesn’t mean both are ready, and my job is to be honest about that against a clear, consistent bar — not to manage it as a popularity problem.

So I’d assess each against the actual criteria for the next level — scope, impact, consistency, how they operate beyond their own tasks — and I’d be able to point to evidence, not gut feel. If both

genuinely meet the bar and I'm constrained by budget or band, I'd advocate hard for both, sequence them transparently, and be clear with each on exactly where they stand and why. If only one is ready, I promote that one — and I don't soften the bar to avoid an awkward conversation, because that's unfair to the person who did clear it and it devalues the promotion.

The part I'd put the most care into is the person who doesn't get it this cycle. I'd tell them directly, own it, show them the specific gap, and give them a concrete path and timeline to get there — so they leave the conversation with a plan, not just a no. How I handle that conversation is what determines whether I keep them."

The structure underneath

5. **Separate expectation from readiness** — assess each against the same explicit bar.
6. **Decide on evidence** — scope, impact, consistency; be able to show your work.
7. **Don't lower the bar** — promoting to avoid awkwardness is unfair to everyone and cheapens the title.
8. **Invest most in the "no"** — direct feedback, specific gap, concrete path and timeline. That conversation is the retention moment.

N3 — "When would you reach out to the Program Manager for help?"

WHAT THEY'RE REALLY TESTING

Clean escalation judgment. Two failure modes they're screening for: the manager who **hides risk** and absorbs it silently until it explodes, and the one who **escalates everything** and can't solve their own problems. You want to land in the middle: escalate early, with a clear ask, for the right things.

Say it like this

"My rule is simple: I escalate early when something is outside my span of control, and I escalate with a recommendation, not just a problem. I don't escalate things I should be solving myself, and I don't sit on a risk hoping it resolves — silent risk is the thing that actually burns programs.

Concretely, I'd reach out to the Program Manager when: a dependency outside my team is going to slip and threaten the timeline; priorities across teams are colliding and need a call above my level; I need resourcing or a scope decision I can't make alone; or a risk has crossed from 'I'm managing it' to 'this could affect the program's commitments.' In every one of those, I come with the situation, what I've already done, the options, and the specific decision or unblock I need — so I'm making their job easier, not handing them a problem.

The principle is that escalating early and cleanly is a strength. The manager who quietly absorbs risk to look self-sufficient is the one who delivers the nasty surprise late — and that's the opposite of what a program needs."

THE LINE THAT LANDS

"I **escalate early, with a recommendation** — not late, with an excuse." Pair it with: "Silent risk is what burns programs; clean escalation is what protects them."

Part C — Rewritten core answers (the three that were backfiring)

Your originals leaned too hard on “I already do this” and put the IC-instinct worry front and center, three times over. These are re-centered on the team and spread your vulnerabilities out.

Q1 — “Why this role, why now?”

WHAT WAS WRONG

Original was three sentences all about *your* growth (“the level I want to grow into”). A Portfolio Manager hears “wants the title.” Lead with the team and the mission; make your motivation the second half.

“This team is at the point where it needs a manager fully focused on its delivery and its growth — not splitting attention with an IC workload. I’ve been carrying both for two years, and I’ve seen what the team can do when someone’s genuinely clearing their path and investing in them. I want to do that properly, with the accountability that comes with the title.

It’s the right moment because the feature-store foundation is in place and the team is ready to scale — and I care about these people’s careers; I grew most of them. The honest personal piece is that people leadership is the work I find most meaningful, and I want to own the outcomes, not just the technical decisions.”

Q3 — “First formal management role — why are you ready, and what worries you?”

WHAT WAS WRONG

Original named the player→coach trap vividly (“resisting the urge to step in,” “I’d be lying if I said it’s automatic”) and made the IC worry the **headline** of your candidacy. That hands the interviewer the exact doubt they were probing. Reframe: **evidence first, worry small and already in hand**. And don’t reuse this worry for Q12.

“I’m ready because I’ve effectively been running this team for two years — the sprint planning, the people conversations, the stakeholder alignment — and the delivery record and the team I’ve grown speak for themselves. The title makes the accountability formal, which I want.

What changes is owning the formal growth and performance conversations end-to-end, and I’ve been deliberately building that muscle — *[one concrete example: e.g., ‘I’ve handed off code reviews I’d normally do, and I’ve put engineers in front of stakeholders where I’d usually present’]*. The thing I hold myself accountable for is staying out of the weeds enough to grow the team — and the way I check myself is simple: if I’m the one writing the fix, I’ve already failed at the part of this job that actually matters.”

THE DIFFERENCE

You still name the IC tension — that *is* mature — but it’s now a footnote to “I’ve led this team for two years,” not the load-bearing wall. Vivid **action** language replaces vivid **doubt** language.

Q12 — “What will be the hardest part of this role for you?”

WHAT WAS WRONG

Original gave the **same** answer as Q3 (letting go of the IC instinct). Three questions, one tripled weakness. Pick a **different** honest difficulty so the leader leaves with a fuller picture, not one flaw in triplicate.

“The honest answer is calibration — knowing how hard to push each person versus when to protect their bandwidth. With a team I grew, I know them well, but as their manager I’ll be making real calls about stretch, promotion readiness, and workload, and early on I won’t have a full year of that data on everyone.

So my plan is to over-index on listening at the start — more 1:1s, more asking, fewer assumptions — and to be willing to adjust quickly when I get a read wrong. I’d rather be slightly too careful early and earn the right to push harder than to misjudge someone’s capacity in week three.”

KEEP Q9 (DELIVERY PHILOSOPHY) ALMOST VERBATIM

“Predictability beats heroics. Make the work — and the trade-offs — visible. Invest in the team’s craft alongside delivery.” It’s your strongest line. Add one earned detail: a time heroics burned you, so it reads as learned, not memorized.

Part D — Questions to ask the Portfolio Manager

Pick 4–5. One question that shows portfolio-level thinking moves you more than three safe ones. Don't ask two that are near-duplicates.

Strong — ask these

- **Opener:** “Where's the biggest gap between where this team is today and where you want it to be?” — *surfaces the real problem they're hiring you to solve.*
- **Portfolio signal (your best question):** “The feature store is reusable infrastructure — are there teams beyond this one you'd want consuming it? Where's the appetite to make it a shared asset?” — *proves you think at portfolio level and reframes your work as org leverage.*
- **The elephant, named:** “This team was my peer group until now. From where you sit, what would make that transition succeed or fail?” — *clear-eyed about the actual hard part; disarms it by naming it.*
- **Protect/change:** “What's one thing the current setup does well that I should protect, and one thing you'd want changed?” — *signals you won't bulldoze what works.*
- **Closer:** “If we're doing my first review a year from now, what would make you say this hire was a clear win?” — *gets you their scorecard. Strong final note.*

Reworded (don't use the original phrasing)

- Instead of “what keeps you up at night” (a cliché senior leaders have heard 50 times), ask: **“Across the streams you oversee, where do you see the most delivery risk right now — and where could this team take work off your plate?”** Same strategic signal, fresher, and positions you as someone who reduces their load.

Cut or downgrade

- Don't ask both “success in 12 months” *and* the “first review” closer — they're the same question; asking both reads as a script. Pick the closer.
- “What's the support structure for a first-time manager?” — re-centers your inexperience. Ask **only** late, and only if the room's gone well and you want logistics. Never lead with it.

Part E — Tonight & day-of

THE SINGLE MOST LIKELY REASON YOU DON'T GET THIS — AND THE FIX

Most likely failure: you show up as a brilliant analytics engineer applying for a manager job, and the Portfolio Manager can't picture you running the team without being in the code. Your stack is the JD, so the room's gravity pulls toward technical territory — and every minute there is a minute you're impressive at the wrong thing.

The fix is one move, repeated: pull every answer back to people. Lead with “I grew this team 1 to 11.” When a question drifts technical, give the short version and steer back to what it meant for the team. If they remember one thing about you, make it that you build people.

Tonight — in priority order, time-boxed. Don't do everything.

9. **(10 min) Lock your four stories in the new ratio:** Built-the-team (1→11), Led-through-tech-shifts, Snowflake-vs-SageMaker conflict, + Feature Store in reserve. Say the first one out loud twice.
10. **(10 min) Fill the two brackets that matter most:** the hard-feedback moment (B2) and the concrete example in Q3. Even a rough real instance beats a polished blank.
11. **(10 min) Read Part B once more** — underperformer, peers→reports, exec update, budget. Just hold the *structures*, not scripts.
12. **(5 min) Internalize the through-line:** “I've led this team two years; the job is to make them excellent without me in the code.”
13. **Then stop and sleep.** You cleared round 1. Over-prep makes you stiff.

During the interview

- Sip water before answering — a 2-second pause that forces the headline out first.
- Use “Are you more interested in the technical side or the people side?” liberally — it lets you choose the people angle.
- End answers cleanly. After the outcome + insight, **stop**. Silence is fine.
- If you start rambling: “Let me bring that back to the point — the headline is X.” Self-correcting reads as maturity.
- Notepad keyword cue: “**Built 1→11 / Tech-shifts / SageMaker-conflict / Feedback / Underperformer / Exec-update.**” Scan, pick, go.

One-line reminders: Headline first. People before tech. Three actions, not seven. Concede merit in disagreement. End and stop.